

An in-depth analysis of key ML concepts in light of the Runge function

Marie Océane Pilskog, Synne Sigstad Jørgensen and Muhammad Mohsin Mumtaz
(Dated: October 6, 2026)

Runge’s phenomenon is widely studied in the field of machine learning, describing how polynomial interpolation leads to oscillations at the edges of the function definition interval. The function is an excellent basis for investigating machine learning concepts such as linear regression methods, optimal polynomial degrees and bias-variance trade-off.

Throughout our analysis, we present key machine learning concepts, with the Runge function as our starting point. We implement the following linear regression methods *OLS*, *Ridge*, and *Lasso*, and further make use of a set of resampling methods *bootstrap* and *k-fold cross-validation*, and plain and stochastic gradient descent. Lastly, we test out a variety of optimisers mentioned hereafter, *Momentum*, *AdaGrad*, *RMSprop*, and *Adam*.

Employing these various methods provides provisional results across our report. Finally, combining the concepts investigated throughout our analysis, we use k-fold cross-validation to find the optimal polynomial degrees. First for OLS we find these degrees $\in [8, 8, 11]$ for k-fold values $\in [5, 10, 25]$. Secondly for Ridge, we obtain the optimal degrees $\in [6, 6, 6]$ for k-fold values $\in [5, 10, 25]$ with respective optimal penalty parameters $\lambda \in [0.0433, 0.0486, 0.1385]$. And lastly, for Lasso, we find the optimal polynomial degree 6 for k-fold value 5, with $\lambda = 0.000599$. Finally, we are able to display the oscillating behaviour associated with the Runge function.

I. INTRODUCTION

Machine learning (ML) is a rapidly developing field with applications across numerous disciplines, among them medicine, humanities, and technology. Its central aim is to help algorithms learn patterns from data without explicitly having to program rules associated with each specific task [1]. The learned patterns can be used to make predictions or inform decisions on new data.

The Runge function is a classic example used in ML illustrating the limitations of high-degree polynomial interpolation (see Equation 1). When interpolated using equally spaced points, the resulting polynomial will exhibit oscillations near the boundaries of the interval; a behaviour known as Runge’s phenomenon, discovered by Carl Runge in 1901 [2]. The phenomenon demonstrates that increasing the polynomial degree will not necessarily improve accuracy [3].

In this paper, we use the Runge function to investigate how polynomial degree and regularisation affect the predictive performance of polynomial regression models. We will assess the model performance by using a set of resampling methods. We will test out different optimisation methods, comparing their associated convergence and the obtained coefficients. Furthermore, analysing both the prediction errors as well as the shapes of the fitted curves, discussing how the observed regression behaviour relates to Runge’s phenomenon. We will first present the relevant theoretical background for the Runge function analysis in section II. In section III the main methods we have applied in our study will be put forth. The obtained results will be presented and discussed in section IV. We will summarize our main takeaways in section V. The algorithm developed for our Runge function analysis may

be found in the following GitHub repository ¹.

II. THEORY

We start by presenting the previously introduced Runge function

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1]. \quad (1)$$

For a realistic analysis a noise term is added, given by a normal distribution defined as $\mathcal{N}(0, \sigma^2)$, where 0 denotes the mean, and σ corresponds to the standard deviation. The value for σ will be specified throughout our results in section IV. In this theory section, the theoretical foundations for our analysis of the Runge function will be presented. We first introduce the regression methods used in our study, followed by feature scaling and the treatment of the intercept. The bias-variance tradeoff will be explained, and the decomposition of the expected prediction error into bias, variance, and irreducible noise will be derived. Finally, the resampling methods used to assess model performance as well as the optimisation methods used to estimate the model coefficients, will be described.

A. Regression methods

Regression aims to model the relationship between a response variable and a set of input features. From observations of the features and their corresponding responses, a model is fitted to approximate the underlying function and predict responses for the new data [1, Chapter 3.1].

¹ <https://github.com/synnesjor/FYS-STK4155>

In this paper, we will consider three regression methods: the Ordinary Least Squares (OLS), Ridge, and Lasso regression methods.

Ordinary Least Squares estimates the model coefficients by minimising the sum of the squared residuals. Each residual is defined as the difference between an observed response and the corresponding model predictions [1, Chapter 3.3]. Provided that the columns of the design matrix are linearly independent, the $\hat{\theta}_{\text{OLS}}$ coefficients are given by the closed-form solution

$$\hat{\theta}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}, \quad (2)$$

where $\mathbf{X} \in \mathbb{R}^{n \times p}$ is the design matrix for n observations and p features, $\mathbf{y} \in \mathbb{R}^n$ describes the function outputs from our discrete data points, and the superscript T denotes the transpose.

Ridge regression extends OLS by adding the regularisation term $\lambda \|\theta\|_2^2$ to the least-squares cost function θ_2^2 . The obtained Ridge coefficients may be derived from

$$\hat{\theta}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}, \quad (3)$$

where $\mathbf{I}$ denotes the identity matrix and λ determines the strength of the penalty. The described penalty will discourage large coefficients and may reduce the variance of the fitted model [1, Chapter 3.8].

Lasso regression interchanges the ℓ_2 -norm penalty used in Ridge regression for a ℓ_1 penalty applied to the least-square cost function. This penalty can set coefficients to exactly zero, meaning it can perform variable selection. The coefficients do not generally have a closed-form solution and must therefore be obtained by numerical optimisation [1, Chapter 3.9]. The cost function for Lasso regression is given by

$$C(\theta) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\theta\|_2^2 + \lambda \|\theta\|_1, \quad (4)$$

where n determines the number of observations and $\lambda \geq 0$ controls the strength of the regularisation. For non-zero coefficients, differentiating the cost function and setting the gradient to zero gives

$$2\mathbf{X}^T \mathbf{X}\theta + n\lambda \text{sgn}(\theta) = 2\mathbf{X}^T \mathbf{y}, \quad (5)$$

where $\text{sgn}(\theta)$ is a vector containing all the signs of θ . The ℓ_1 penalty is, however, is not differentiable at zero, meaning that the above expression does not cover zero-coefficients, which will require special treatment during optimisation.

For Ridge and Lasso, the described penalties will depend on the magnitude of the coefficients, which in turn are affected by the scales of the features. OLS, however, will be exempt as Equation 2 is equivariant under rescaling of the columns. This further implies that the fitted values will not change, in contrast to the coefficients, which will do so in a compensating way. In the cases of Ridge and Lasso, expressing distances in kilometers

rather than meters, for example, would require a coefficient a thousand times larger to produce the same predictions, thereby increasing its contribution to the penalty. Standardizing the features resolves the described unit dependence [1, Section 3.13]. Each non-constant feature may be standardised using

$$x_j^{(i)} \rightarrow \frac{x_j^{(i)} - \bar{x}_j}{\sigma(x_j)}, \quad (6)$$

where i denotes the observation, j denotes the feature, and $\bar{x}_j$ and σ_j are the feature's training mean and standard deviation. The obtained features will have a zero mean and unit standard deviation. The constant intercept column should be excluded as it carries a zero standard deviation.

The intercept determines the baseline level of the predictions and should not be penalised. The response values are centred in the following manner,

$$y_c^{(i)} = y^{(i)} - \bar{y}, \quad (7)$$

where $y_c^{(i)}$ is the centered y -value for an observation, $y^{(i)}$ is the original observation, and $\bar{y}$ is the mean. When both the features and responses are centred, the intercept of the centred model is zero. The model can then be fitted without an intercept column. Predictions on the original response scale are obtained by adding the training response mean $\bar{y}$ back to the model's predictions [1, Section 3.13].

It should be specified that the means and standard deviations used for these transformations will be calculated from the training data only.

B. Prediction quality: mean squared error and R^2

To properly evaluate the predictive performance on a given data set, the mean squared error (MSE) and coefficient of determination R^2 are used. MSE measures the average squared difference between the observed and predicted target values, given by [1, Chapter 3.5]

$$\text{MSE} = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2. \quad (8)$$

Here, n is the number of observations, y_i is the observed target value, and $\tilde{y}_i$ is the corresponding model prediction. A smaller MSE will be indicative of a more accurate prediction.

Since MSE uses the squared units of the target, the R^2 is used as a dimensionless measure of the prediction quality. R^2 essentially compares the model's squared prediction error with the one obtained by predicting the mean target value [1, Chapter 3.5]. The R^2 -score may be defined as

$$R^2 = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}, \quad (9)$$

where

$$\bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i \quad (10)$$

describes the mean target value on the evaluated data set. A score of $R^2 = 1$ is indicative of a perfect prediction, whereas $R^2 = 0$ is equivalent to the same squared error as predicting the mean. Lastly, $R^2 < 0$ indicates an even worse performance.

C. The bias-variance trade-off

Having introduced the regression methods and explained how scaling, centering, and the intercept should be handled, let us now consider how well the models predict new observations. The predictive performance of a model depends not only on how closely they fit the training data, but additionally on the change in predictions across different training data sets. The bias-variance decomposition is a framework for understanding the sources of prediction error and the contribution of irreducible noise.

1. Bias, variance, and irreducible error

Bias is the difference between the expected value of an estimator and the true parameter value. Mathematically, this may be written as

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta, \quad (11)$$

where $\hat{\theta}$ is an estimator of the true parameter θ , and the expectation is taken over a set of different samples [1, Chapter 2.4]. The same principle applies to model predictions. Prediction bias corresponds to the difference between the average prediction over training sets and the true function value. A model that is too simple to capture the underlying relationship can have a high bias prediction and underfit the data. Lower prediction bias means that the average prediction is closer to the true function value, but does not necessarily imply low prediction error, as *variance* must also be considered. Variance measures how much an estimator changes when it is computed from a set of different samples and may be written as [1, Chapter 2.6]

$$\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]. \quad (12)$$

For model predictions, high variance indicates sensitivity to a particular training set. A flexible model may fit noise in the training data, which can cause the predictions to vary substantially between training sets and thereby increase the risk of overfitting. In this case, MSE is defined from combining the bias and variance as follows

$$\text{MSE} = \mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Bias}(\hat{\theta})^2 + \text{Var}(\hat{\theta}). \quad (13)$$

The *irreducible error* σ^2 describes the variance of random noise in the observations. When the noise has zero mean and is independent of the training data, it contributes σ^2 to the expected squared prediction error and cannot be eliminated by improving the model. Further implying, that the expected test MSE will be given by

$$\text{Expected test MSE} = \text{Bias}^2 + \text{Variance} + \sigma^2. \quad (14)$$

The *bias-variance tradeoff* describes how model complexity affect the bias and variance. Simpler models tend to associate with higher bias and lower variances. More flexible models on the other hand tend to carry lower bias and higher variance as they are more sensitive towards the training data. The aim is to find a model that minimises the combined prediction error. Regularisation may reduce variance, but will also introduce bias. In the following section, the bias-variance decomposition will be derived.

2. Derivation of the bias-variance decomposition

Assume that the observations are generated following

$$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2).$$

Let $\tilde{y}$ denote the model prediction at a fixed input x , which will depend on the training set used to fit the model. The expected squared prediction error will be derived by taking the expectation over different training sets and the noise from an independent test observation. Hereafter we use $f = f(x)$ and start by adding and subtracting $\mathbb{E}[\tilde{y}]$ as follows

$$\mathbb{E}[(y - \tilde{y})^2] = \mathbb{E}[(f + \varepsilon - \tilde{y})^2] \quad (15)$$

$$= \mathbb{E}[(f - \mathbb{E}[\tilde{y}] + \mathbb{E}[\tilde{y}] - \tilde{y} + \varepsilon)^2]. \quad (16)$$

Expanding the square gives

$$\begin{aligned} \mathbb{E}[(y - \tilde{y})^2] &= \mathbb{E}[(f - \mathbb{E}[\tilde{y}])^2] + \mathbb{E}[(\mathbb{E}[\tilde{y}] - \tilde{y})^2] + \mathbb{E}[\varepsilon^2] \\ &\quad + 2\mathbb{E}[(f - \mathbb{E}[\tilde{y}])(\mathbb{E}[\tilde{y}] - \tilde{y})] \\ &\quad + 2\mathbb{E}[\varepsilon(f - \mathbb{E}[\tilde{y}])] \\ &\quad + 2\mathbb{E}[\varepsilon(\mathbb{E}[\tilde{y}] - \tilde{y})]. \end{aligned} \quad (17)$$

The first three terms correspond to squared bias, prediction variance, and noise variance. Since f and $\mathbb{E}[\tilde{y}]$ are fixed at the chosen input,

$$\mathbb{E}[(f - \mathbb{E}[\tilde{y}])^2] = (f - \mathbb{E}[\tilde{y}])^2 = \text{Bias}(\tilde{y})^2. \quad (18)$$

The second term corresponds to

$$\mathbb{E}[(\mathbb{E}[\tilde{y}] - \tilde{y})^2] = \text{Var}(\tilde{y}). \quad (19)$$

Since the noise has zero mean, we may write

$$\mathbb{E}[\varepsilon^2] = \text{Var}(\varepsilon) + \mathbb{E}[\varepsilon]^2 = \sigma^2. \quad (20)$$

The remaining three terms vanish. For the first cross term, seeing as f is not stochastic, we find

$$\mathbb{E}[(f - \mathbb{E}[\tilde{y}])(\mathbb{E}[\tilde{y}] - \tilde{y})] = (f - \mathbb{E}[\tilde{y}]) \underbrace{\mathbb{E}[\mathbb{E}[\tilde{y}] - \tilde{y}]}_0 = 0. \quad (21)$$

For the second cross-term

$$\mathbb{E}[\varepsilon(f - \mathbb{E}[\tilde{y}])] = \underbrace{\mathbb{E}[\varepsilon]}_0 \cdot \mathbb{E}[f - \mathbb{E}[\tilde{y}]] = 0. \quad (22)$$

Finally, the test noise is independent of the training data and therefore of the prediction $\tilde{y}$. Hence,

$$\mathbb{E}[\varepsilon(\mathbb{E}[\tilde{y}] - \tilde{y})] = \underbrace{\mathbb{E}[\varepsilon]}_0 \cdot \underbrace{\mathbb{E}[\mathbb{E}[\tilde{y}] - \tilde{y}]}_0 = 0. \quad (23)$$

Combining these results gives the approximation

$$\mathbb{E}[(y - \tilde{y})^2] \approx \text{Bias}(\tilde{y})^2 + \text{Var}(\tilde{y}) + \sigma^2. \quad (24)$$

For n fixed test inputs, averaging the pointwise decomposition gives

$$\begin{aligned} \mathbb{E} \left[\frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2 \right] &= \frac{1}{n} \sum_{i=1}^n (f(x_i) - \mathbb{E}[\tilde{y}_i])^2 \\ &\quad + \frac{1}{n} \sum_{i=1}^n \mathbb{E}[(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2] + \sigma^2. \end{aligned} \quad (25)$$

Thus, the expected test MSE consists of the averaged squared bias, the averaged prediction variance, and the irreducible noise [1, Chapter 2.10].

If noisy test responses are used instead of the true function values when estimating squared bias, then

$$\mathbb{E}[(y - \mathbb{E}[\tilde{y}])^2] = (f(x) - \mathbb{E}[\tilde{y}])^2 + \sigma^2. \quad (26)$$

This quantity therefore, includes the noise variance. In our study, the Runge function is known, allowing the squared bias to be evaluated directly against the noise-free function values.

D. Resampling methods

The bias-decomposition considers how a model's prediction changes when it has been fitted to different training sets. In practice, however, only one data set is available. Resampling methods repeatedly select observations from the data set to form new training samples. The sensitivity of the model to the training set can, in this manner, be assessed, as well as how the model predicts observations that were not used for fitting. An estimator

calculated based on random data will itself be a random variable. The corresponding sampling distribution will describe how the value changes across different samples. Resampling estimates properties of the distribution without needing the underlying data distribution [1, Chapter 2.11].

The *bootstrap* resampling method constructs new samples by drawing observations with replacement from the original data set. Each bootstrap sample contains the same number of observations as the original sample. It is important to understand that the method implies that some observations may appear numerous times, whereas others may be omitted. The model will be fitted to each sample, with the resulting estimates or predictions being used to assess the variability.

The *jackknife* resampling method constructs samples by systematically leaving out one observation at a time and recalculating the estimator on the remaining observations. Both the bootstrap and the jackknife methods assume independent, identically distributed observations; dependent data would require appropriate adaptations [1, Chapter 2.11].

The *k-fold cross-validation* method splits the data into k approximately equal, non-overlapping folds. Each fold is used once for validation, while the remaining $k - 1$ folds are used to train the model. The validation scores are then averaged to estimate performance on unseen data. Consequently, each observation is used for validation once, and for training $k - 1$ times [1, Chapter 2.12].

E. Optimisers

Optimisers iteratively update the model coefficients to minimise a cost function. Gradient-based methods use the gradient to determine the direction of each update, while the learning rate controls the size. In the Runge function analysis several optimisers will be tested out, each with its own technique influencing the updates. *Plain* gradient descent updates the coefficients of the model in the direction of the negative gradient of the cost function, with the learning rate controlling the step size [1, Chapter 4.3]. The update rule may be defined as

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma_t \nabla C(\boldsymbol{\theta}_t), \quad (27)$$

where $\boldsymbol{\theta}_t$ is the coefficient vector at iteration t , $\gamma_t > 0$ is the learning rate, and $\nabla C(\boldsymbol{\theta}_t)$ is the gradient of the cost function. For this type of quadratic cost function with a positive definite Hessian, a constant learning rate γ will reach convergence when

$$0 < \gamma < \frac{2}{\lambda_{\max}}, \quad (28)$$

where $H = \nabla_{\boldsymbol{\theta}}^2 C$ denotes the Hessian matrix of second derivatives and $\lambda_{\max}$ is the largest eigenvalue [1, Chapter 4.5]. Typically, one would use the 90% upper bound of Equation 28.

Momentum based gradient descent combines the current gradient with information from previous updates. The optimisation method may accelerate progress in consistent directions and reduce oscillations during optimisation [1, Chapter 4.6]. The analytical function for the momentum-based gradient descent is given by

$$\begin{aligned}\mathbf{v}_{k+1} &= \beta \mathbf{v}_k + \gamma \nabla C(\boldsymbol{\theta}_k), \\ \boldsymbol{\theta}_{k+1} &= \boldsymbol{\theta}_k - \mathbf{v}_{k+1}.\end{aligned}\quad (29)$$

Here, $\boldsymbol{\theta}_k$ corresponds to the parameter vector, C is the cost function, $\mathbf{v}_k$ is the previous accumulated update, γ is the learning rate, and $\beta \in [0, 1)$ is the momentum coefficient, typically 0.9.

AdaGrad adapts the effective learning rate for each coefficient using the accumulated sum of its squared gradients. Coefficients with larger accumulated squared gradients receive smaller effective learning rates. The rates may become very small as the optimisation proceeds [1, Chapter 4.9]. The analytical function describing the AdaGrad-based gradient descent is given by

$$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \circ \mathbf{g}_t. \quad (30)$$

The parameter update is further defined as

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma H_t^{-1/2} \mathbf{g}_t. \quad (31)$$

In individual coordinates, this becomes

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\gamma}{\sqrt{r_{t,j}}} g_{t,j}, \quad (32)$$

or, with a numerical stability constant,

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\gamma}{\sqrt{\varepsilon + r_{t,j}}} g_{t,j}. \quad (33)$$

Here, $\mathbf{g}_t$ is the current gradient, $\mathbf{r}_t$ accumulates squared gradients from $\mathbf{r}_0 = \mathbf{0}$, $\circ$ denotes elementwise multiplication, and $H_t = \text{diag}(\mathbf{r}_t)$. The index j identifies a parameter, γ is the learning rate, and $\varepsilon > 0$ prevents division by zero.

RMSprop uses an exponentially weighted average of squared gradients instead of AdaGrad's cumulative sum. Recent gradients will as such, get greater weights, avoiding the continual accumulation which risks making AdaGrad's updates too small [1, Chapter 4.10]. The analytical update rule for RMSprop is given by

$$\mathbf{v}_t = \rho \mathbf{v}_{t-1} + (1 - \rho) (\nabla C(\boldsymbol{\theta}_t))^2, \quad (4.47)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\gamma}{\sqrt{\mathbf{v}_t + \varepsilon}} \nabla C(\boldsymbol{\theta}_t). \quad (4.48)$$

Here, $\mathbf{v}_t$ is an exponentially weighted average of squared gradients, ρ is the decay coefficient, typically 0.9 or 0.99, γ is the learning rate, and $\varepsilon > 0$ is a numerical stability constant. Squaring, square roots, and division are

elementwise. For RMSprop, $\mathbf{v}_t$ corresponds to $\mathbf{r}$ in the code.

Adam combines an exponentially weighted average of gradients with an exponentially weighted average of squared gradients. It moreover corrects the initial bias in the averages, combining momentum with an adaptive effective learning rate for each coefficient [1, Chapter 4.11]. The analytical update rule for Adam parameters may be given by

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t + \varepsilon}} \hat{\mathbf{m}}_t. \quad (34)$$

Here, $\mathbf{m}_t$ is a moving average of the gradient, whereas the $\mathbf{v}_t$ is the elementwise square. The initialisation was done using $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$. The hats are indicative of bias correction. The learning rate α corresponds to `gamma` in the code, $\mathbf{v}_t$ corresponds to `r`, and $\varepsilon > 0$ prevents division by zero.

III. METHODS

In this section, we will present the main algorithms used for the analysis of Runge function. We will not present all the algorithms included in our code, instead we have chosen to put forth the methods developed in the most important ones. We would like to acknowledge that, despite not all being explicitly mentioned hereafter, we have used and often modified the algorithms presented by Morten Hjort-Jensen in the course material for FYS-STK3155/FYS-STK4155 [4] to help us develop functioning algorithms. Pseudo-code will be presented for the regression algorithms we have developed ourselves.

We have also used `scikit.learn`, a Python library providing tools for machine learning and statistical programming [5]. From this library, we utilise the built-in functions for Ridge regression, OLS regression (which is the same as Ridge but with $\lambda = 0$), calculation of MSE and R^2 values, cross-validation, and resampling, amongst many others.

A. Implementation of regression algorithms

Algorithm 1 illustrates how the Ridge gradient function is built. Note that when the penalty parameter λ is zero, the function is equivalent to the OLS gradient function. For Ridge, the penalty term is the ℓ_2 -norm.

Algorithm 2 describes the algorithm we have used for estimating the Lasso regression cost function, as given in **Equation 4**. One may notice that for Lasso regression, the ℓ_1 -norm is used rather than the ℓ_2 -norm in the penalty term (as used in **Algorithm 1**), implying that there will be no closed form solution as the term will not be differentiable at zero.

Algorithm 1 Ridge Gradient Function

Require:

θ $\triangleright$ Parameter vector (polynomial coefficients)
 X $\triangleright$ Rescaled design matrix up to the given degree
 n $\triangleright$ Number of data points
 y $\triangleright$ Target values, length n
 $\lambda \geq 0$ $\triangleright$ Regularisation strength; $\lambda = 0$ gives OLS

Do:

$r \leftarrow X\theta - y$ $\triangleright$ Residuals
 $g_{\text{MSE}} \leftarrow -\frac{2}{n}X^T r$ $\triangleright$ Gradient of the mean squared error
 $g_{\text{L2}} \leftarrow 2\lambda\theta$ $\triangleright$ Gradient of the L2 penalty
return $g_{\text{MSE}} + g_{\text{L2}}$

Algorithm 2 Lasso Gradient Function

Require:

θ $\triangleright$ Parameter vector (polynomial coefficients)
 X $\triangleright$ Rescaled design matrix up to the given degree
 n $\triangleright$ Number of data points
 y $\triangleright$ Target values, length n
 $\lambda \geq 0$ $\triangleright$ Penalty

Do:

$r \leftarrow y - X\theta$ $\triangleright$ Residuals
 $g_{\text{MSE}} \leftarrow -\frac{2}{n}X^T r$ $\triangleright$ Gradient of the mean squared error
 $g_{\text{L1}} \leftarrow 2\lambda \text{sign}(\theta)$ $\triangleright$ Subgradient of the L1 penalty
return $g_{\text{MSE}} + g_{\text{L1}}$

B. Optimiser functions

For the optimiser functions, we have borrowed Morten Hjort-Jensen's functions from [6]: `optimiser_step` and `optimiser`, which can be found in the `utils.py` script in our code. The function `optimiser_step` updates θ , i.e., the polynomial coefficient from a gradient g for one step t , employing Equation 27-34 presented in section II E. The equations describe how the update happens for the following optimiser methods: Plain, Momentum, Ada-Grad, RMSprop and Adam. The function `optimiser` then runs the `optimiser_step` function, saving the corresponding θ -values, and returning the number of iterations needed for convergence within a defined tolerance. For cases where the tolerance is never reached, `optimise` will run through all the iterations until reaching a previously defined maximum iteration number. Both the tolerance and the number of iterations vary throughout our code, and will be specified when presenting our corresponding results.

C. Stochastic gradient descent

Our minibatch stochastic gradient descent algorithm is called `stochastic_gradient_descent` and may be found in the `utils.py` file of our project. The function implementation has been inspired by [6]. To implement the SGD-algorithm, all parameters are first set to zero. Plain gradient descent computes the gradient from the

full data set at every step. SGD however, instead estimates it from a small random subset which in this context will be called a minibatch. At the start of each epoch, the data indices are shuffled and split into minibatches. For each minibatch, we estimate the OLS or Ridge gradient using the associated rows of the design matrix and target values. The parameters are then updated with a chosen optimiser, described in section II E. The learning rate is either constant or decays following a schedule $\gamma_t = t_0/(t + t_1)$. Here, t counts all updates so far, t_0/t_1 sets the initial learning rate, and t_1 controls how quickly the decay happens. A decaying learning rate will dampen the noise in the minibatch gradients so the parameters can settle at the minimum. After each epoch, we check how much the parameters have changed: the run stops early if the change is smaller than a chosen tolerance (convergence) or larger than 10^2 (divergence). The function returns the parameters after each epoch, which we use to study the convergence of our analysis.

D. Specifics associated with Lasso implementation

As will be seen later in section IV, for Lasso regression, we struggle to find convergence for several of the optimisers. To make sure that our implementation of the Lasso regression algorithm is correct, we have executed a set of tests, which will be described henceforth. First, the Lasso algorithm was run with a penalty parameter $\lambda = 0$, to simulate the equivalence to the OLS case. The results could then be compared with the previously obtained OLS polynomial coefficients (θ). The obtained θ -values are identical, meaning that there is nothing wrong with our implementation of the MSE term of the cost function. The Lasso function was then tested again, but now with the ℓ_2 norm instead of the ℓ_1 norm used for Lasso. If the rest of the function is implemented correctly, we expect to see the same θ -values as for our Ridge code. Again, we obtain the expected values corresponding to our Ridge results. These tests further imply that, should there be a remaining error in our implementation, it must be associated with the penalty term. After closely examining the term, we believe that it has been implemented correctly and continue with the Lasso analysis.

For OLS and Ridge, we first stopped the gradient descent when the norm of the gradient fell below a defined tolerance of $\|\nabla C(\theta)\| < \varepsilon$. The approach works as the corresponding cost functions are differentiable, meaning that the gradient goes smoothly to zero as we approach the minimum. For Lasso, the defined criterion was repeatedly never met, and the optimisers continued to run until the maximum number of iterations was reached. This may be explained by the ℓ_1 penalty $\lambda\|\theta\|_1$, which is not differentiable at $\theta_j = 0$. It is therefore replaced by the subgradient $2\lambda \text{sign}(\theta)$, which does not shrink as we approach the minimum. Coefficients that should be zero, will instead oscillate around zero. The gradient norm therefore stays of order λ and never drops be-

low ε , even when θ is essentially at the minimum. We therefore changed the stopping criterion to be defined based on the change in the parameters between iterations, $\|\theta_t - \theta_{t-1}\| < \varepsilon$, which will go below the set tolerance once θ has stopped changing noticeably from one iteration to the next.

IV. RESULTS AND DISCUSSION

A. Ordinary least squares: mean square error, coefficient of determination, and polynomial degree

We start with the straightforward OLS case, where we have performed a singular value decomposition to solve Equation 2, so we can find the mean square errors, as given in Equation 13, and the R^2 score, as given in Equation 9–10. Figure 1 shows the MSE and R^2 -values for polynomial degrees ranging from 1 to 15. For the training data, we find from the MSE values that the optimal polynomial degree is 14, while for the test data, we find a polynomial degree of 7. This difference is expected, as for the training MSE values, we train our model on a certain set of noise values, and the higher the polynomial, the better the fit for that specific noise data set. However, when we use the trained model to fit the test data with new noise data points, the model is fitted so closely to the noise data points from the training data that it may no longer be a good fit for the test data. This is what we call *overfitting*, and demonstrates that the highest polynomial degree may not necessarily be the best one. The MSE curves in Figure 1 are compared to Figure 2.11 in [7]. We, however, observe that we do not get the smooth curves we wish for. We also see that the discrepancy between the training and test MSE are much smaller for our data compared to the results described in [7]. There can be many contributing factors, among them the choice of the number of data points n , and the standard deviation σ for the noise.

From Figure 1 we infer agreement between the R^2 -values and the MSE, giving us the same optimal polynomial degree for both training and test data, with 14 and 7 respectively. The negative value for the first polynomial degree on the test data, implies that the model is worse than the mean for this specific degree, as previously stated in section II B.

Figure 2 displays how the value for θ varies with the polynomial degree for our test data. For the lower degrees, we see that the values do not vary much as we increase the number of polynomial degrees in our model. However, around a degree of 7 and onward, we see that the values start to vary to a greater extent. This agrees with our earlier findings, that the optimal polynomial degree for our model trained on our data is 7. Beyond this degree, we begin overfitting to the noise in the training data.

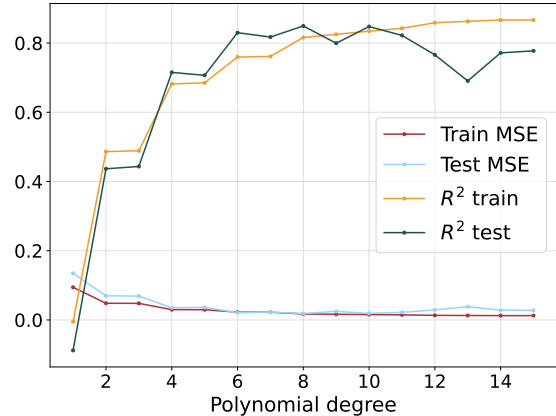


FIG. 1. Mean square error and R^2 values, for testing and training data. The maximum R^2 gives us the optimal polynomial degree for our function, likewise with the minimum MSE. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, and number of data points $n = 100$. The training and test data is split 80/20.

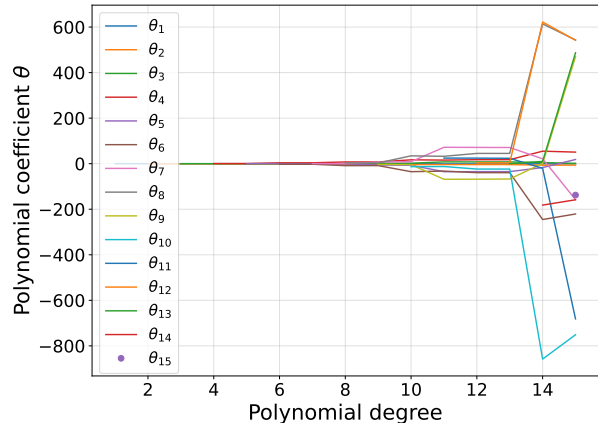


FIG. 2. Polynomial coefficients θ_i as a function of polynomial degree i . Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, and number of data points $n = 100$.

B. Ridge Regression

Figure 3 displays how the values for θ for a 5th degree polynomial varies with the penalty parameter λ . As expected, the lowest value for λ reproduces values for θ close to the values found with OLS regression (as we approach $\lambda = 0$). The scales on the y -axis for Figure 2 and Figure 3 makes this difficult to see, but it has been confirmed in our algorithms.

From Figure 3, we infer that θ_2 and θ_4 are first affected by λ , followed by θ_3 , and θ_5 . θ_1 seems to be least affected by changes in the λ -values. When the penalty becomes

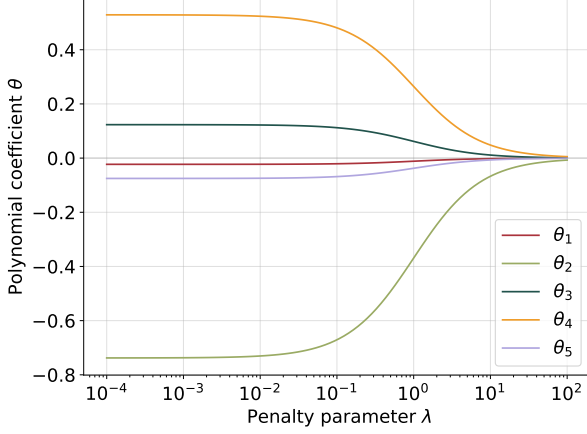


FIG. 3. Polynomial coefficient θ_i as a function of penalty parameter λ . Each curve represents a coefficient $i \in [1, 5]$. The penalty parameters are in the range $\lambda \in [10^{-4}, 10^2]$. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, and number of data points $n = 100$.

very large, the values for all coefficients converge towards 0.

C. The bias-variance tradeoff and the bootstrap

Figure 4 shows the MSE decomposition, as explained in section II C 2, using OLS. Here, the *bootstrap* re-sampling technique has been used, as explained in section II D, using 100 bootstraps. It is clear that for the lower polynomial degrees, the bias (which is plotted together with the variance on the noise σ^2) dominates, while the variance is small. At these smaller polynomial degrees one can clearly see the tradeoff between the two phenomena; as the bias decreases, the variance increases. This is what we would expect, as a higher number of polynomial degrees increases the model's sensitivity to the random noise of the training data, which increases the variance, and at the same time we also expect a lower bias, as we are making our model more complicated by increasing the polynomial degrees. Figure 4 shows the expected tradeoff (with some small variations, but the numbers we are working with are quite small, so it is not unexpected) up to a polynomial degree of about 8, with the variance overtaking the bias at a degree of 7. From a polynomial degree 9, one can start to see an increase in both bias and variance (again, with some variations). When both variance and bias is increasing, it is clear that the model is no longer a very good one, as it is both too sensitive to the random noise of the training data, as well as the model being too simple: a lose-lose situation.

From Figure 4 it is deduced that the total test error is the lowest at a polynomial degree of 6, and although this error does not alone decide the best fit, it is a good

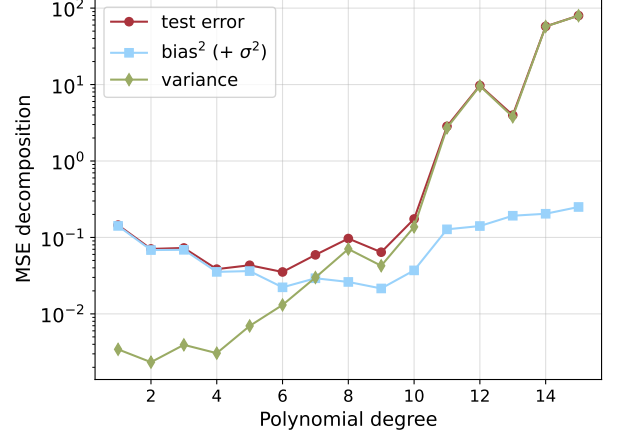


FIG. 4. MSE decomposition components and total test error as functions of polynomial degree. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, and number of data points $n = 100$.

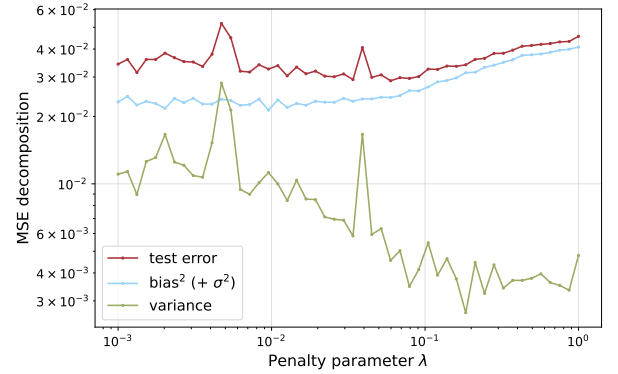


FIG. 5. MSE decomposition components and total test error as functions of the penalty parameter λ , at a polynomial degree 6. The penalty parameters are in the range $\lambda \in [10^{-3}, 1]$. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, and number of data points $n = 100$.

indicator. This agrees well with the previous OLS result of degree 7 obtained without bootstrapping. The total expected test error also includes the irreducible noise variance σ^2 , in addition to squared bias and variance. Subtracting this constant contribution might contribute to lowering the error curve and change the obtained optimal degree. However, the results from this investigation is only one polynomial degree away from the earlier results for the optimal polynomial degree of 7.

Figure 5 shows the bias-variance decomposition using the Ridge method. Here, the MSE is plotted against the penalty parameter λ , using a polynomial degree of 6, as this is the polynomial degree where the lowest test error for the OLS case was found. For the Ridge case the minimum MSE is found at $\lambda \approx 0.0596$.

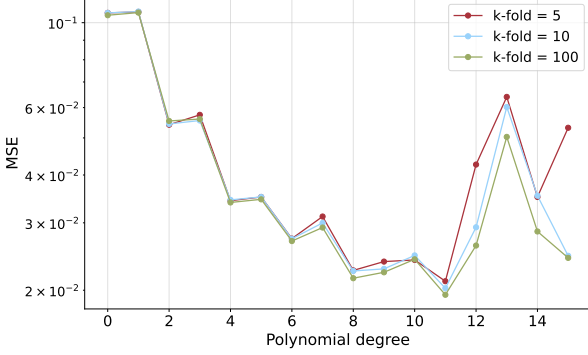


FIG. 6. MSE as function of polynomial degree, using OLS regression the k-fold cross-validation resampling technique. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, and number of data points $n = 100$.

D. k-fold cross-validation and its comparison to the bootstrap approach

In Figure 6 we have plotted the MSE as a function of the polynomial degree for OLS, for k-folds 5, 10, and 100. Because we use $n = 100$ data points, when we have a k-fold of 100, this is the leave-one-out case. For all values for the k-fold, the minimum MSE is found at a polynomial degree 11. This differs from our earlier results, where 6 and 7 were found. The difference does not improve when increasing the number of observations, which may indicate an issue with our execution of the cross-validation. The discrepancy could be due to whether we choose to use the whole data set, or split it into test and train data. In Figure 6 we did not split the data set, whilst in the earlier Figure 4 we did. Less data contributes to a higher variance, which in turn drives the model away from higher polynomial degrees, as the variance becomes the main component of the MSE. We should see this problem reduced in Ridge and Lasso regressions in the later sections, as it penalises this particular aspect of the MSE, and keeps the variance low even for high polynomial degrees. Figure 7 shows the Ridge MSE as a function of the penalty parameter λ , for a polynomial degree 6. It is now possible to compare values for λ with our results by looking at the earlier bias-variance tradeoff. The resulting penalty parameters are $\lambda \approx 0.0215$, $\lambda \approx 0.0272$, and $\lambda \approx 0.0343$ for the k-fold values 5, 10, and 100 respectively. For the latter — the “leave-one-out”-case — the value for λ is the closest to the value we found when investigating the bias-variance tradeoff for Ridge. There still however is a certain difference. Multiple reason may be found for this discrepancy: the number of bootstraps, the number of k-folds, choice of polynomial degree to investigate, or a combination of these. For example, one can see that if we change the number of bootstraps to 1400 in section IV C and leave everything else as it, the value for the penalty parameter

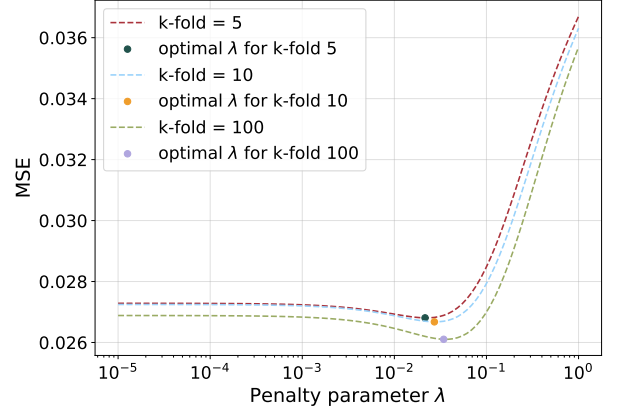


FIG. 7. MSE as function of penalty parameter λ , using Ridge regression the k-fold cross-validation resampling technique. The penalty parameters are in the range $\lambda \in [10^{-5}, 1]$. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, and number of data points $n = 100$.

is now $\lambda \approx 0.0391$, which is closer to the value from the cross-validation approach.

E. Gradient Descent

Now, the analytical expressions for the optimal parameters θ for OLS and Ridge will be replaced by our own written gradient descent algorithm. We are now encountering methods of regression where there are many parameters and even more ways to combine them, so one has to pick out a selection of combinations to test.

For OLS, the learning rates $\gamma = 0.1$, $\gamma = 0.3$, and $\gamma = 0.5$ were tested. For a polynomial degree 6, and a tolerance of 10^{-6} (tolerance defined as the norm of the gradient at each iteration), 9773 iterations are needed for convergence at $\gamma = 0.1$, and 29327 are needed for convergence at $\gamma = 0.3$. For a learning rate $\gamma = 0.5$, it diverges, meaning that the steps are too large to converge towards a minima.

For Ridge, the same learning rates are tested, each with a small selection of the optimal lambdas deduced earlier in this section: $\lambda = 0.0343$ and $\lambda = 0.0596$. One smaller and two larger values are also tested for curiosity: $\lambda = 0.01$, $\lambda = 0.1$, and $\lambda = 0.3$. The number of iterations for the combinations of λ and γ are presented in Table I. The table shows that the combination of the two parameters is important. Although the number of iterations for $\gamma = 0.3$ seem to decrease for increasing values for λ , as well as having the lowest number of iterations, at one point the number of iterations starts to increase again, and at $\lambda = 0.3$, a learning rate $\gamma = 0.1$ gives the lowest number of iterations.

When working with the gradient descent, it is also interesting to investigate the gradient itself, and the differ-

TABLE I. Resulting number of iterations from different combinations of the penalty parameter λ , and the learning rate γ . The values in the leftmost column are the values for λ , and the values in the top row are the values for γ .

	0.1	0.3	0.5
0.01	4535	1508	diverged
0.0343	1603	531	diverged
0.0596	984	324	diverged
0.1	618	202	diverged
0.3	223	716	diverged

ence in the gradient between different types of differentiation. Here we use analytical differentiation, which is the combined gradient of the mean squared error and ℓ_2 penalty (for Ridge) described in Algorithm 1, as well as automatic differentiation, using JAX [8]. The difference in the gradient between the two is astonishingly low for both OLS and Ridge; $\sim 2.22 \times 10^{-16}$ for both, or in other words it is accurate to machine precision.

F. Momentum and more advanced ways to update the learning rate

In the context of the optimiser algorithms presented in section II E, the number of iterations needed for convergence and the learning rate γ have been investigated. Some of the optimisers do not work very well with the OLS approach, as can be seen in Figure 8. Only Momentum, AdaGrad, and Adam converge for multiple learning rates, while Plain only converges for one, whose value is comparable to the learning rate calculated from Equation 28. Both plain and adagrad only converges when we use a large number of iterations, like 100000 as used in Figure 8. If we go down to 10000 iterations, the two do not converge for any of the chosen learning rates. RMSprop does not converge for any of the tested learning rates, regardless of running for a large number of iterations.

For Ridge with $\lambda = 0.1$, we obtain the results shown in Figure 9 when the number of iterations needed for convergence is plotted against the learning rate γ . For a sufficiently large λ -value, all optimiser methods converge. For a lower value for λ , such as the earlier found $\lambda = 0.0343$, the RMSprop does not converge at any learning rate, shown in Figure 10. However, one can see that for a penalty parameter of $\lambda = 0.1$, the RMSprop is actually the fastest to converge for the learning rates where it converges. Adam is the method that converges for the largest range of learning rates, as seen for both $\lambda = 0.1$ and $\lambda = 0.0343$ in Figure 9 and Figure 10 respectively.

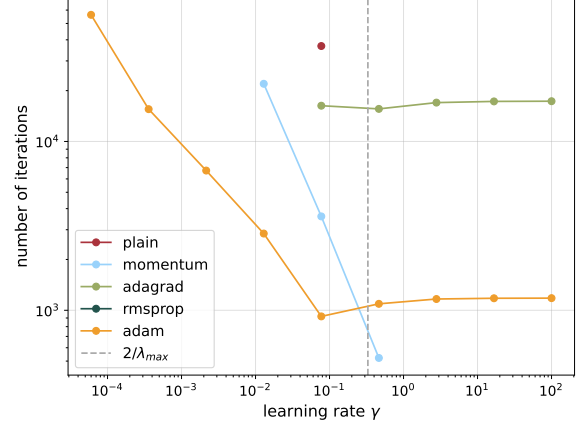


FIG. 8. Number of iterations as a function of learning rate γ , for the optimising algorithms Plain, Momentum, AdaGrad, RMSprop, and Adam. For a polynomial degree 6, and 100000 iterations. The learning rates are in the range $\gamma \in [10^{-5}, 10^2]$. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

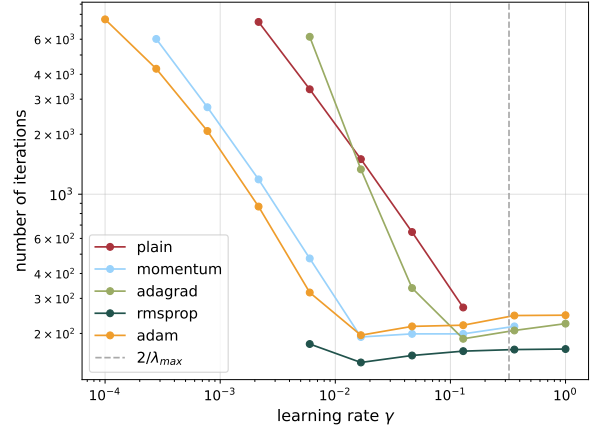


FIG. 9. Number of iterations as a function of learning rate γ , for the optimising algorithms Plain, Momentum, AdaGrad, RMSprop, and Adam. For a polynomial degree 6, a penalty parameter $\lambda = 0.1$, and a maximum of 100000 iterations. The learning rates are in the range $\gamma \in [10^{-4}, 1]$. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

G. Lasso regression

Lasso regression, compared to the previous two models, is much more sensitive to the chosen learning rate. For plain descent in Figure 11, we allowed for 100000 iterations, and for high enough learning rates, such as $\gamma = 10^{-4}$, we see that the loss function for Lasso is significantly reduced. The visual results for Lasso in Fig-

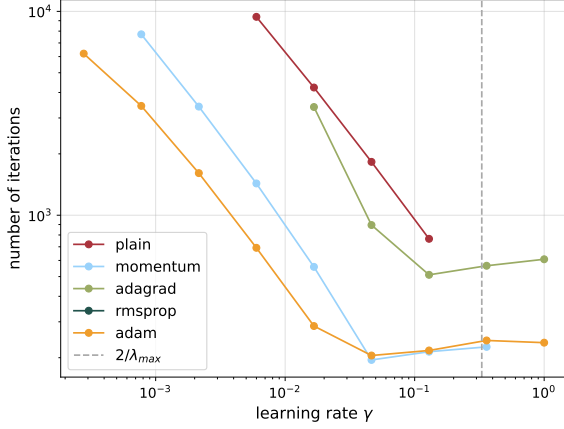


FIG. 10. Number of iterations as a function of the learning rate γ , for a a penalty parameter $\lambda = 0.0343$. The learning rates are in the range $\gamma \in [10^{-4}, 1]$, and we run for a maximum of 100000 iterations. For this combination of parameters, the RMSprop method does not converge at any of the chosen learning rates. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

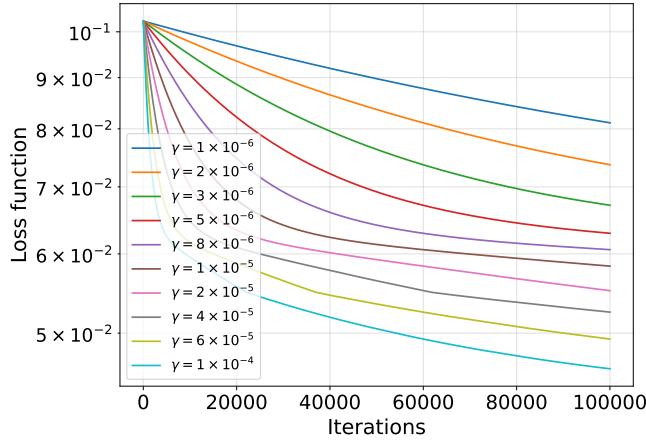


FIG. 11. Number of iterations as a function of the learning rate, given different learning rates from $\gamma = [10^{-6}, 10^{-4}]$ using Plain descent. The penalty parameter is $\lambda = 0.01$, and we use a maximum of 100000 iterations. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

ure 11–15 are different from the convergence tests for Ridge and OLS precisely because we lack analytical solutions to compare the parameters to. As previously described in section IIID, an attempt to find convergence by looking at the change in the norm of the parameters θ was made. For very low learning rates, convergence was reached fast. However, for higher learning rates with better optimised loss functions, convergence could not be obtained.

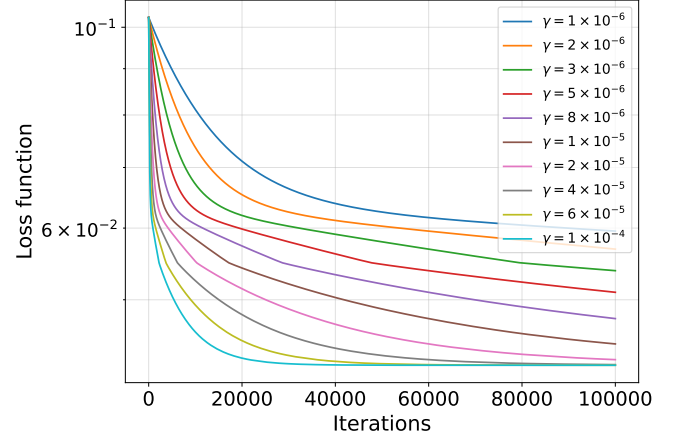


FIG. 12. Number of iterations as a function of the learning rate, given different learning rates from $\gamma = [10^{-6}, 10^{-4}]$ using optimiser method Momentum. The penalty parameter is $\lambda = 0.01$, and we use a maximum of 100000 iterations. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

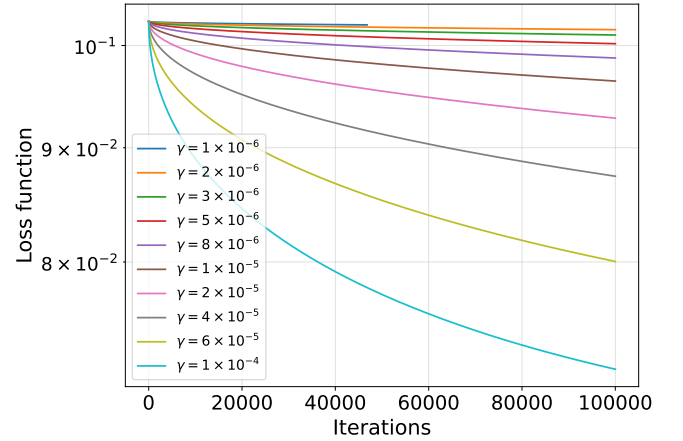


FIG. 13. Number of iterations as a function of the learning rate, given different learning rates from $\gamma = [10^{-6}, 10^{-4}]$ using optimiser method AdaGrad. The penalty parameter is $\lambda = 0.01$, and we use a maximum of 100000 iterations. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

Figure 11 at the very least confirms that the Lasso regression model is functional, and does optimise its respective loss objective as was tested and assumed. However, it is a difficult task to compare this model to the others, and it is only in the model selection section IV I that we might see the difference.

Figure 13, displaying Lasso regression for the AdaGrad optimiser, shows some interesting features. For very low learning rates, the change in parameters would be so low that our algorithm would reach convergence after very few iterations, despite this graph having the worst learn-

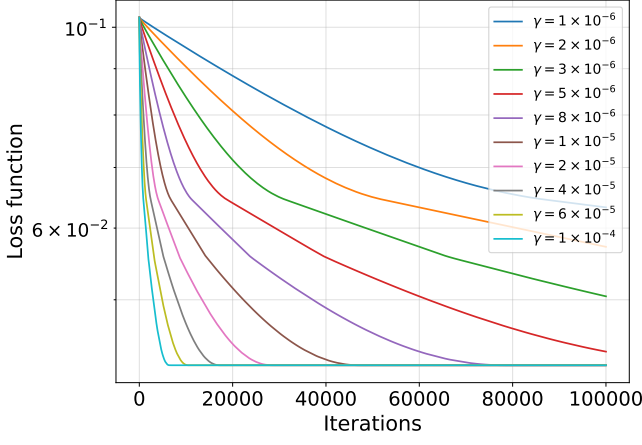


FIG. 14. Number of iterations as a function of the learning rate, given different learning rates from $\gamma = [10^{-6}, 10^{-4}]$ using optimiser method RMSprop. The penalty parameter is $\lambda = 0.01$, and we use a maximum of 100000 iterations. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

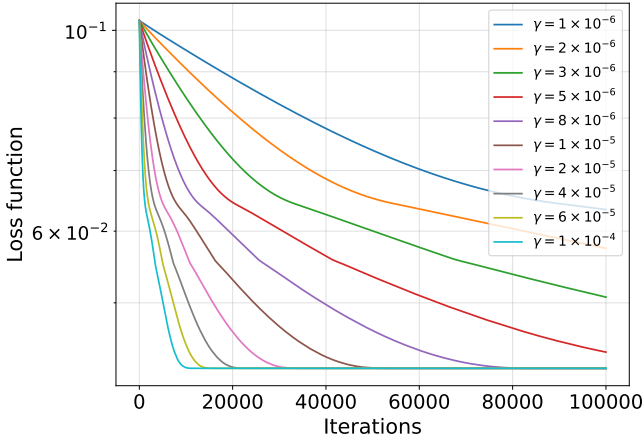


FIG. 15. Number of iterations as a function of the learning rate, given different learning rates from $\gamma = [10^{-6}, 10^{-4}]$ using optimiser method Adam. The penalty parameter is $\lambda = 0.01$, and we use a maximum of 100000 iterations. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

ing rate in optimising the loss function. This could be a direct consequence of our tolerance definition stated in [section III D](#).

For RMSprop in [Figure 14](#), the loss function hits a boundary for minimising the loss function at around 3×10^{-2} . This happens for all learning rates that lead to convergence before the maximum number of iterations is reached, here set to 100000 iterations. The same applies for the Momentum optimiser in [Figure 12](#) and the Adam optimiser in [Figure 15](#). They all reach the same minimum value. Regardless of which method is used, it seems we are able to reach a similar solution when given

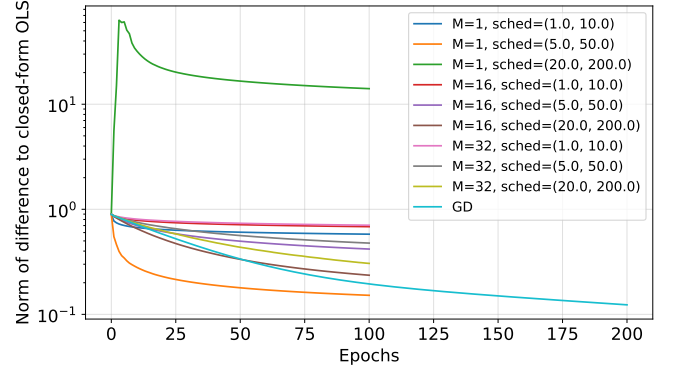


FIG. 16. Norm difference to the closed-form solutions for OLS as a function of the epochs for Plain stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient descent, the maximum number of epochs is 100, and the learning rate used in the standard gradient descent is found through [Equation 28](#). Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

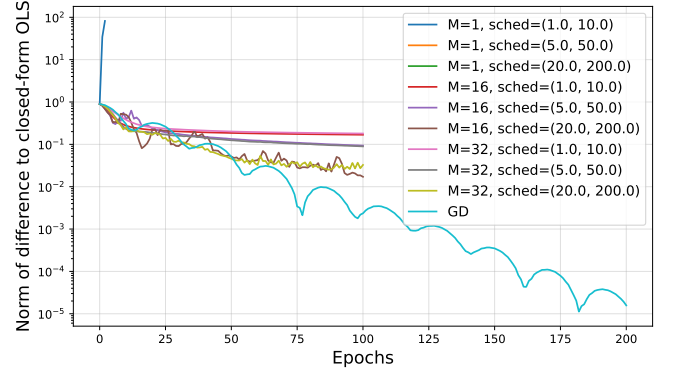


FIG. 17. Norm difference to the closed-form solutions for OLS as a function of the epochs for Momentum stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient descent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through [Equation 28](#). Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

enough iterations. The only difference is how fast we are able to reach that solution. Both OLS and Ridge have a lower limit as to how far the loss function can be minimised, due to the variance of the noise in the dataset: the irreducible variance.

H. Stochastic Gradient Descent

For OLS, the Stochastic gradient descent (SGD) consistently performs better than Plain gradient descent for several optimisers as shown in [Figure 16–20](#). For most

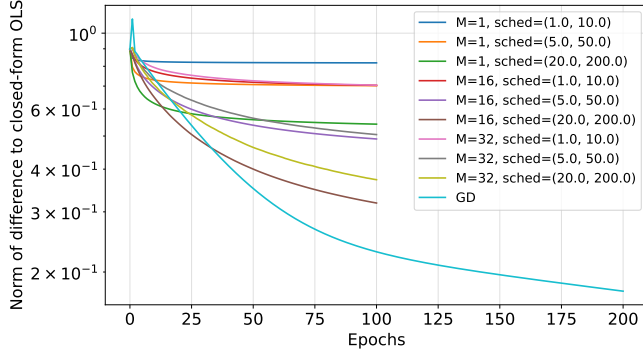


FIG. 18. Norm difference to the closed-form solutions for OLS as a function of the epochs for AdaGrad stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient descent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

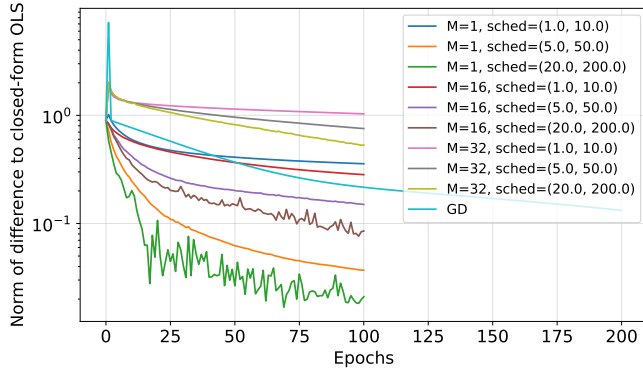


FIG. 19. Norm difference to the closed-form solutions for OLS as a function of the epochs for RMSprop stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient descent, the maximum number of epochs is 100, and the learning rate used in the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

optimisers, there is always one schedule and batch size, which results in a quicker convergence towards the closed form solutions for 100 epochs. AdaGrad and momentum are exempt from the previously described behaviour. When it comes to computational cost, each epoch is roughly equivalent in cost to one iteration of the standard gradient descent methods, even though the amount of updates to the parameters is greater. This is one of the strengths of this method. However, as can be seen in e.g. Figure 20, the noise is much greater, and when run for many epochs, the SGD struggles to reach the closed-form solutions. This is seen in Figure 16, where

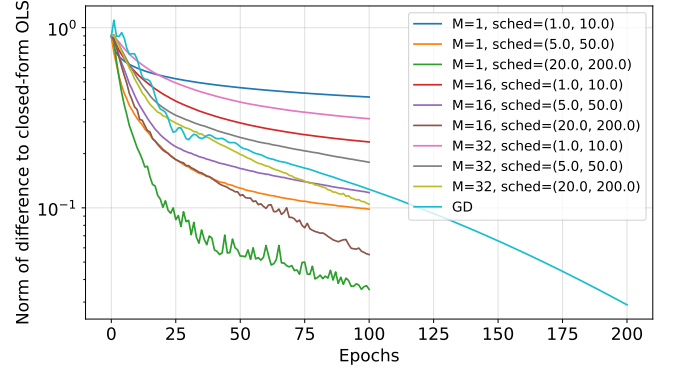


FIG. 20. Norm difference to the closed-form solutions for OLS as a function of the epochs for Adam stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient descent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

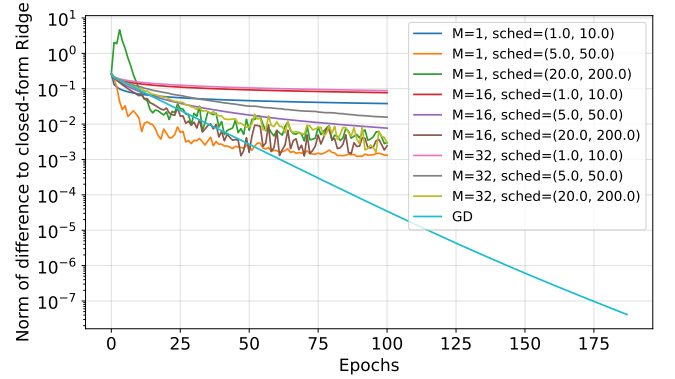


FIG. 21. Norm difference to the closed-form solutions for Ridge regression with penalty parameter $\lambda = 0.01$ as a function of the epochs for Plain stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient descent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

the SGD for various schedules and batch sizes stalls above 10^{-2} , whilst the normal GD slowly converges towards the closed form solution. Overall, for the OLS, it seems that the SGD can optimise the parameters at a quicker pace than the standard gradient descent. OLS carries a smaller computational cost, however, it is too susceptible to noise and struggles to converge. A potential use for the SGD could be to employ it for the first iterations, then switch to standard gradient descent for later iterations.

For Ridge regression, SGD follows a noisier pattern compared to the SGD for OLS, displayed in Figure 21-

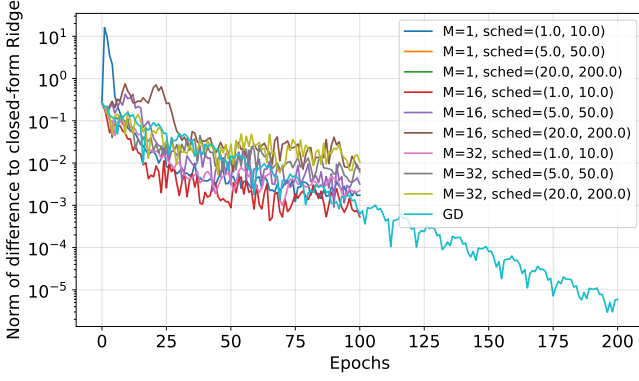


FIG. 22. Norm difference to the closed-form solutions for Ridge regression with penalty parameter $\lambda = 0.01$ as a function of the epochs for Momentum stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient decent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

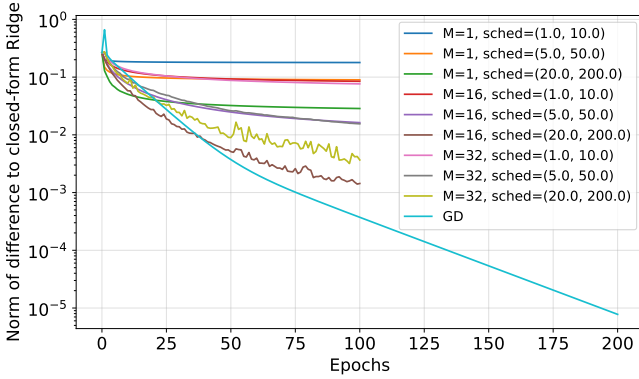


FIG. 23. Norm difference to the closed-form solutions for Ridge regression with penalty parameter $\lambda = 0.01$ as a function of the epochs for AdaGrad stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient decent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

25. For all the optimisers with the different schedules and batch sizes, it seems not worth using the SGD at all, and it is better to run the standard descent method. Ridge regression already optimises for variance at higher polynomial degrees, and therefore, not much room is left for optimisation. Instead, more noise than necessary is added.

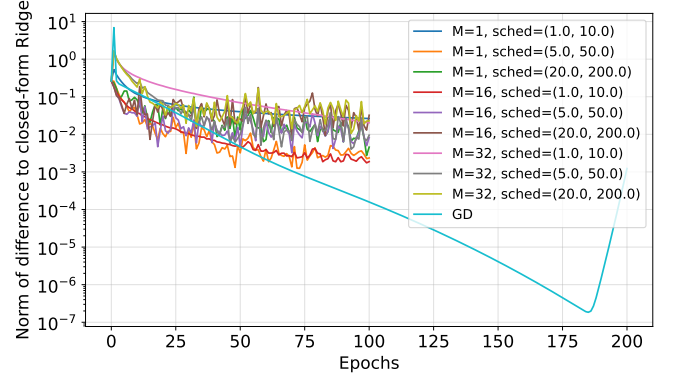


FIG. 24. Norm difference to the closed-form solutions for Ridge Regression with penalty parameter $\lambda = 0.01$ as a function of the epochs for RMSprop stochastic Gradient Descent and as a function of iterations for RMSprop gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient decent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

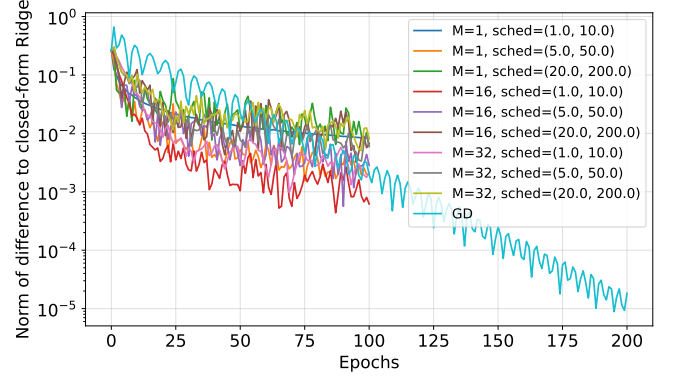


FIG. 25. Norm difference to the closed-form solutions for Ridge Regression with penalty parameter $\lambda = 0.01$ as a function of the epochs for Adam stochastic gradient descent. Different graphs for different schedules and batch sizes. For the stochastic gradient decent, the maximum number of epochs is 100, and the learning rate used for the standard gradient descent is found through Equation 28. Specific parameters used here include standard deviation on the noise $\sigma = 0.1$, a number of data points $n = 100$, and a tolerance $= 10^{-6}$.

I. Model Selection

We first split the data into training and test, and calculate the MSE decomposition for different k-fold values, deducing the optimal OLS-degree for each k-fold value, thereby optimising for the bias-variance tradeoff.

Figure 26–28 display the corresponding variance and bias for each degree and k-fold of the OLS. Table II shows the optimal degree choice for each of these combinations.

Figure 28, displaying the results for a k-fold value of 25

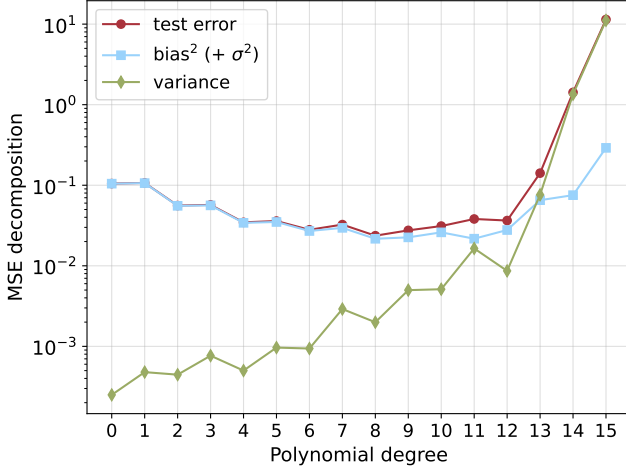


FIG. 26. MSE decomposition components and total test error as a function of the polynomial degree at 5-fold validation for OLS model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

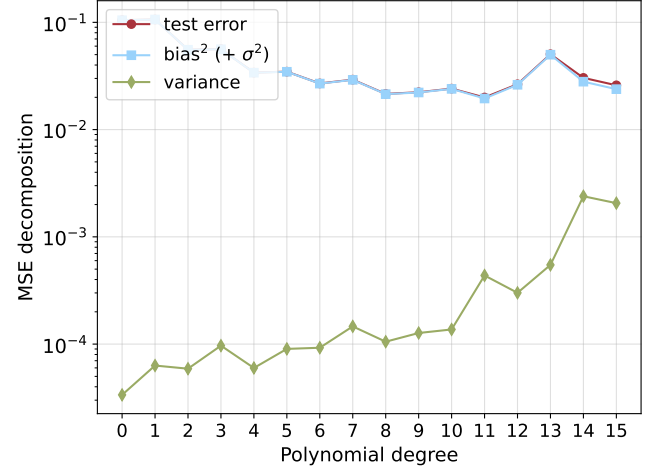


FIG. 28. MSE decomposition components and total test error as a function of the polynomial degree at 25-fold validation for OLS model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

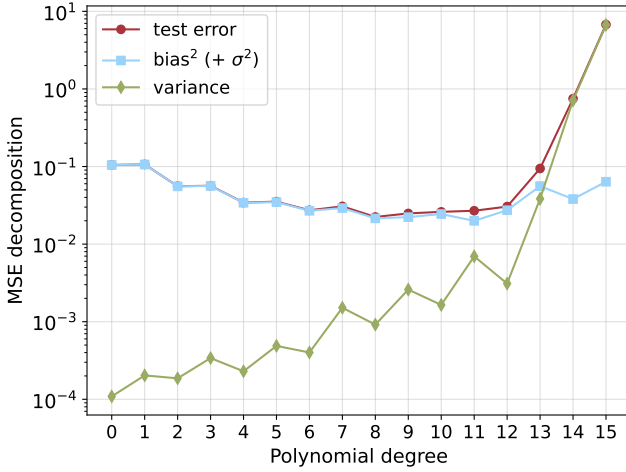


FIG. 27. MSE decomposition components and total test error as a function of the polynomial degree at 10-fold validation for OLS model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

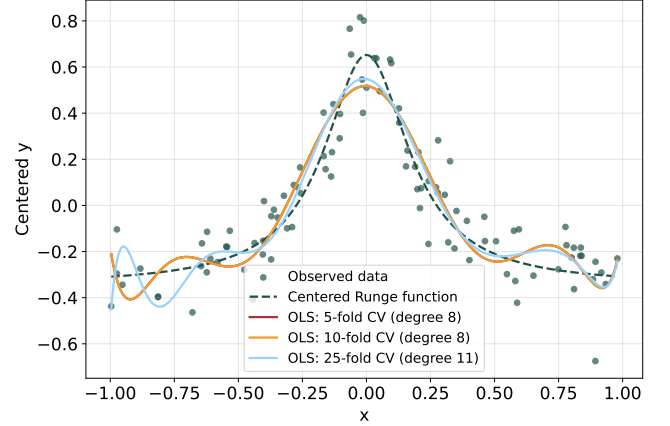


FIG. 29. Estimated solutions for the Runge function for polynomial degrees 8 and 11, using the OLS model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

TABLE II. Optimal polynomial degree and corresponding MSE for different numbers of cross-validation folds for OLS model with noise standard deviation $\sigma = 0.1$, and a number of data points $n = 100$.

	Degree	MSE
5-fold	8	0.0236478
10-fold	8	0.0223992
25-fold	11	0.0199371

looks very different from Figure 26-27, displaying k-fold values 5 and 10. The 5-fold and the 10-fold both have a variance term larger than the bias term at higher polynomial degrees. For the 25-fold validation, the variance term and the bias term have similar values for higher polynomial degrees. A possible explanation may be that a larger part of the training data is used in the higher value k-folds calculation. With more data, the variance should decrease, which in turn allowed for the possibility of a higher fitted polynomial degree.

Also note that in Table II, the higher k-fold (25-fold), which resulted in an optimal polynomial degree of 11, also had a lower MSE value. It can therefore be argued that it is the best solution in the set.

The optimal polynomial degree for the OLS from this analysis is 8 for the 5-and 10-fold, and 11 for the 25-

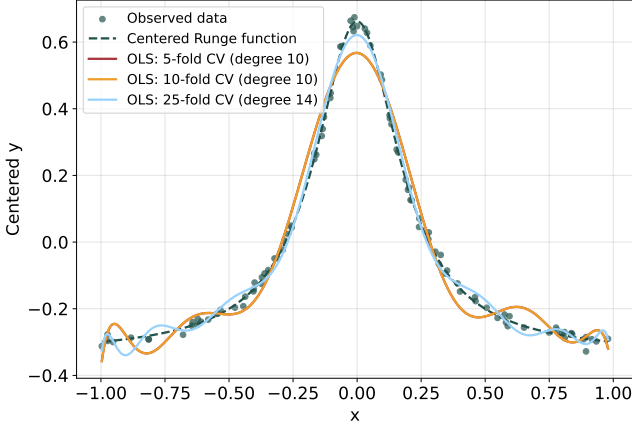


FIG. 30. Estimated solutions for the Runge function for polynomial degrees 10 and 14, using the OLS model with noise standard deviation $\sigma = 0.01$ and number of data points $n = 100$.

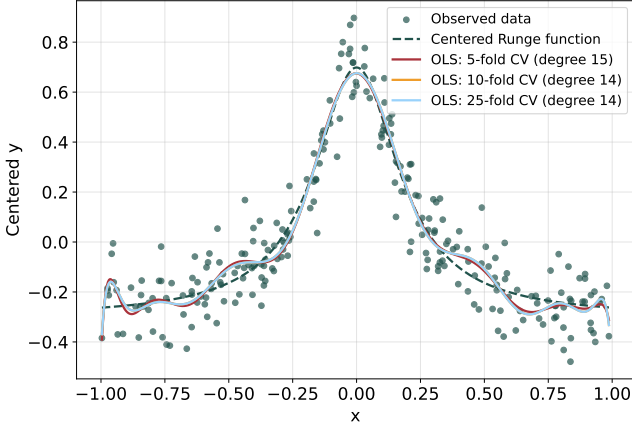


FIG. 31. Estimated solutions for the Runge function for polynomial degrees 14 and 15, using the OLS model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 250$.

fold. These values are for a noise standard deviation level $\sigma = 0.1$. The prediction for the centered Runge function for these degrees is shown in Figure 29.

When reducing the noise standard deviation to $\sigma = 0.01$, the optimal polynomial degrees shift to 10 and 14 as shown in Figure 30.

Another quick test may be to see whether adding more data points will result in a better fit. This is to see whether or not more data points induce overfitting issues as was discussed in the Runge's phenomenon introduction, see section I. The result for an increased number of data points is shown in Figure 31. We notice that adding more points seems to have addressed the aforementioned issues of high variance, and observe that the variance decreases when adding more data points. This furthermore made it possible to select higher polynomial degrees in

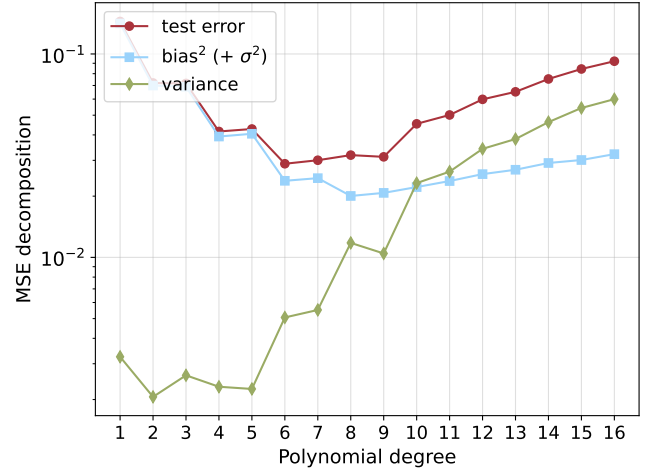


FIG. 32. Ridge regression MSE decomposition components and total test error as a function of the polynomial degree at optimal penalty parameter λ for 5-fold validation with data points $n = 100$ and noise standard deviation $\sigma = 0.1$.

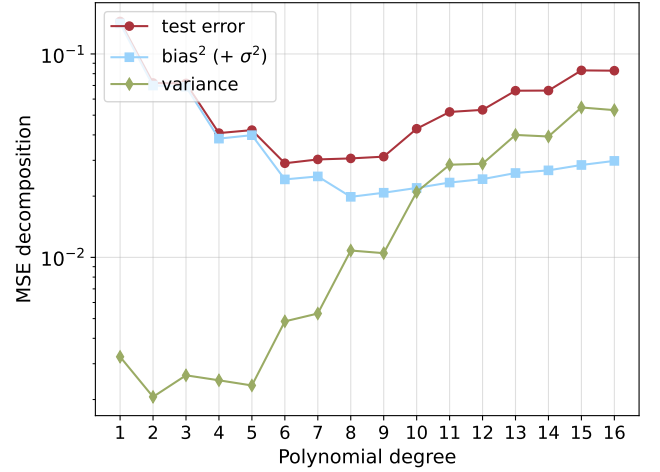


FIG. 33. Ridge regression MSE decomposition components and total test error as a function of the polynomial degree at optimal penalty parameter λ for 10-fold validation with data points $n = 100$ and noise standard deviation $\sigma = 0.1$.

the model selection.

To select the polynomial degree and regularisation strength for approximating the Runge function using Ridge regression, we first identify the optimal penalty parameter for each degree and respective k-fold, and the obtained results are shown in Table III. For each degree and optimal λ , the bootstrap resampling technique is used to search for the optimal degree that minimises the test error for each k-fold.

Figure 32–34 show the corresponding variance and bias for each degree given the optimal penalty parameter for each k-fold of the Ridge regression model, and Table IV shows the optimal choice for degree.

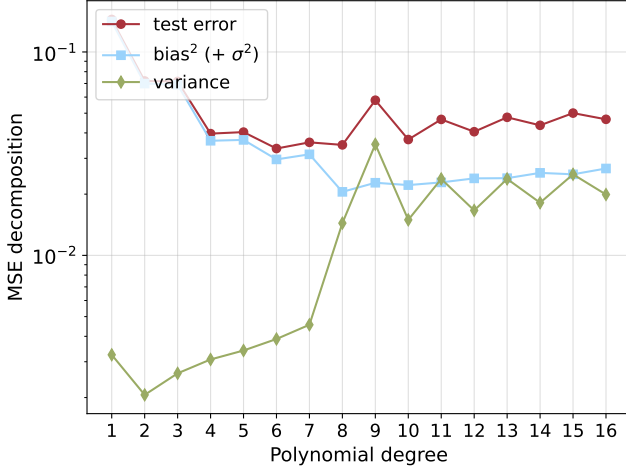


FIG. 34. Ridge regression MSE decomposition components and total test error as a function of the polynomial degree at optimal penalty parameter λ for 25-fold validation with data points $n = 100$ and noise standard deviation $\sigma = 0.1$.

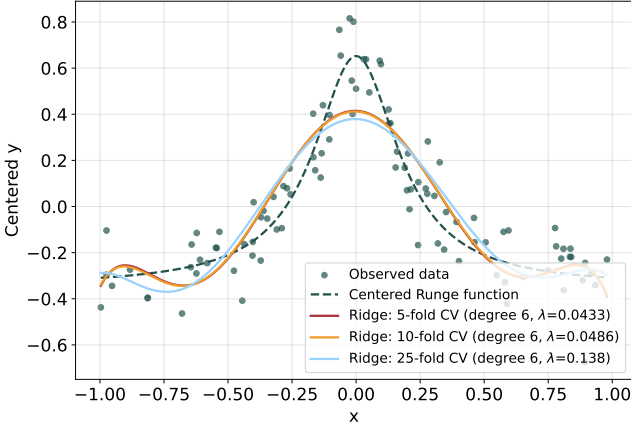


FIG. 35. Estimated solutions for the Runge function for polynomial degree 6, using the Ridge regression model with number of data points $n = 100$ and noise standard deviation $\sigma = 0.1$.

In the MSE decompositions, we may observe that for the different k-folds in the Ridge regression model, a higher overall variance in higher polynomials is driving the test error, which forces all three k-folds to select lower polynomial degrees to account for the bias-variance tradeoff, and instead focus on optimising for the choice of the penalty parameter. This is illustrated in Figure 35 which shows the estimated solutions for the Runge function using the Ridge regression model with noise standard deviation $\sigma = 0.1$ plotted for the optimal degree of 6.

In Figure 36, the estimated solution for the Runge function for Ridge regression, finding an optimal degree of 8, is shown. Here, the noise has been set to $\sigma = 0.01$, and the penalty parameters are once more varied. We may observe that the chosen degree is higher for lower

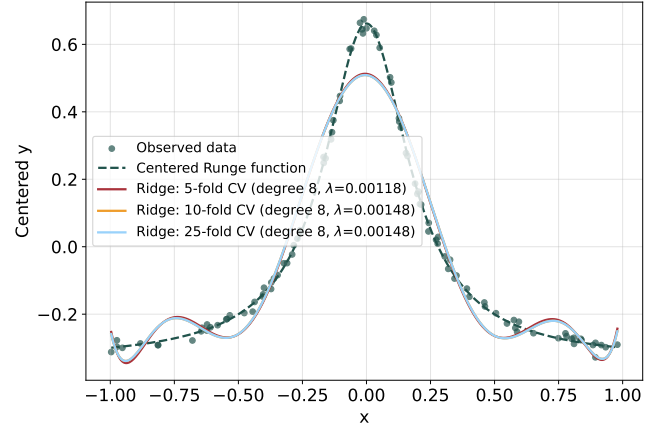


FIG. 36. Estimated solutions for the Runge function for polynomial degree 8, using the Ridge regression model with number of data points $n = 100$ and noise standard deviation $\sigma = 0.01$.

TABLE III. Ridge regression, optimal values of the penalty parameter λ for different polynomial degrees and numbers of cross-validation folds. The values in the leftmost column are the polynomial degrees, and the values in the top row are the number of folds used for cross-validation. Total data points $n = 100$ and noise standard deviation $\sigma = 0.1$.

Degree	5-fold	10-fold	25-fold
1	1.0	1.0	1.0
2	1.0	1.0	1.0
3	1.0	1.0	1.0
4	0.7925	0.6280	0.2783
5	1.0	0.8902	0.3126
6	0.04329	0.04863	0.1385
7	0.05462	0.06136	0.1748
8	2.915×10^{-4}	5.214×10^{-4}	1.000×10^{-5}
9	9.326×10^{-4}	1.048×10^{-3}	1.000×10^{-5}
10	5.722×10^{-5}	1.150×10^{-4}	7.391×10^{-4}
11	4.037×10^{-5}	6.428×10^{-5}	5.857×10^{-4}
12	1.292×10^{-4}	2.915×10^{-4}	2.105×10^{-3}
13	1.292×10^{-4}	2.057×10^{-4}	1.874×10^{-3}
14	2.595×10^{-4}	5.214×10^{-4}	4.229×10^{-3}
15	2.915×10^{-4}	4.642×10^{-4}	3.765×10^{-3}
16	4.642×10^{-4}	7.391×10^{-4}	6.734×10^{-3}

noise. This could, however, be expected as the optimisation focuses on replicating the Runge function, rather than fitting to the outliers of the noisier model seen in Figure 35.

From the values in Table IV, we see that the penalty parameters for the 5-fold and 10-fold results in a significantly lower MSE value than the one found for the 25-fold results. Looking closer at the actual solutions in

TABLE IV. Ridge regression, optimal polynomial degree, corresponding penalty parameter λ , and MSE for different numbers of cross-validation folds. Number of data points $n = 100$ and noise standard deviation $\sigma = 0.1$.

	Degree	λ	MSE
5-fold	6	0.04329	0.028858
10-fold	6	0.04863	0.0290163
25-fold	6	0.13849	0.0335167

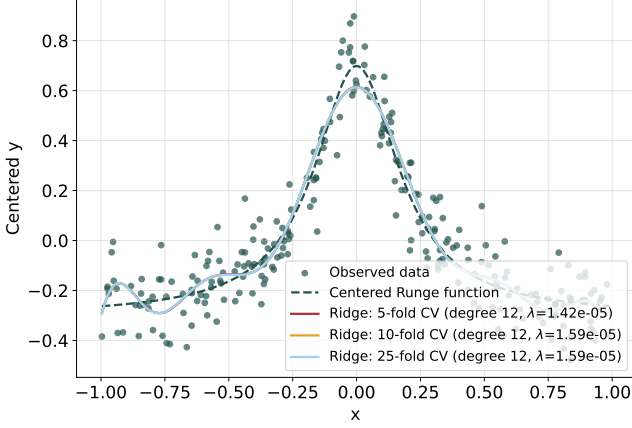


FIG. 37. Estimated solutions for the Runge function for polynomial degree 12, using the Ridge regression model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 250$.

Figure 35, the first two fit closer to the points at the maximum of the Runge function. It may also be said that even though the solutions present with different λ -values, the three curves remain quite similar both in shape and values.

Figure 37 illustrates what will happen when increasing the number of data points. The new solutions all have a lower MSE as shown in Table V, as well as choosing higher polynomial values and lower penalty parameters. With more points, the variance is also lower, and the MSE is mostly driven by the bias rather than the variance, shown in Figure 38, which allows for higher polynomials as previously discussed.

The Lasso regression analysis of the Runge function

TABLE V. Optimal polynomial degree, corresponding penalty parameter λ , and MSE for different numbers of cross-validation folds with noise standard deviation $\sigma = 0.1$ and number of data points $n = 250$.

	Degree	λ	MSE
5-fold	12	1.417×10^{-5}	0.0155328
10-fold	12	1.592×10^{-5}	0.015612
25-fold	12	1.592×10^{-5}	0.015612

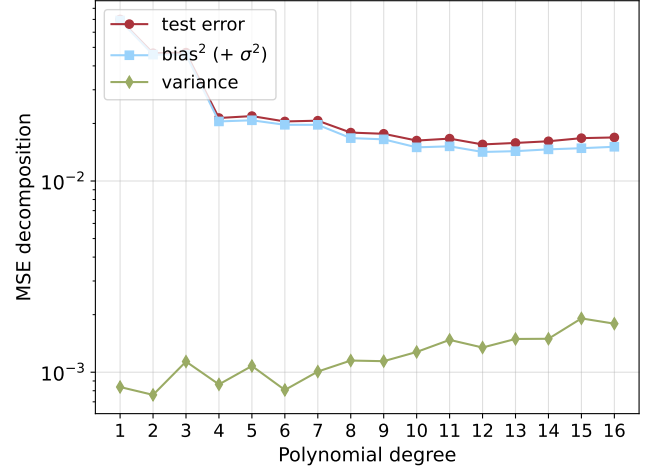


FIG. 38. Ridge regression MSE decomposition components and total test error as a function of the polynomial degrees at optimal penalty parameter λ for 5-fold validation with noise standard deviation $\sigma = 0.1$ and number of data points $n = 250$.

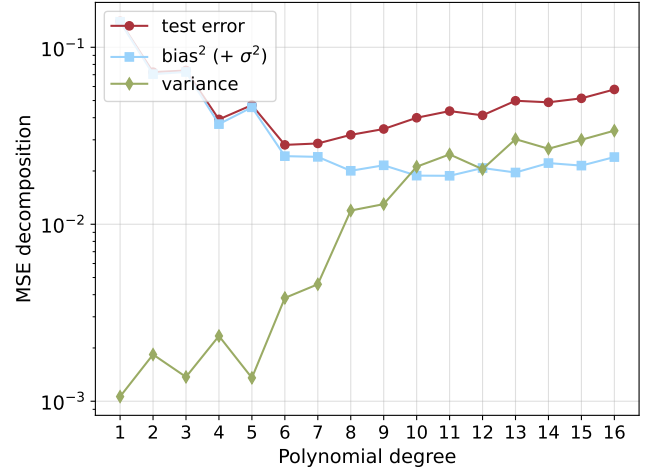


FIG. 39. Lasso regression MSE decomposition components and total test error as a function of the polynomial degrees at optimal penalty parameter λ for 5-fold validation with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

follows the same procedure as the Ridge regression analysis. However, only 5-fold cross-validation is used for Lasso because of the computational cost of using more folds. First the penalty parameter is optimised for each degree, the results may be found in Table VI. The optimal degree is selected in the same manner as for OLS and Ridge by balancing the variance and bias. The obtained results may be seen in Figure 39, and Table VII. When comparing these results to the ones presented for OLS at 5-fold in Figure 26, the variance seems to follow a similar pattern of staying low for low polynomial degrees and increasing for higher degrees. Unlike for the

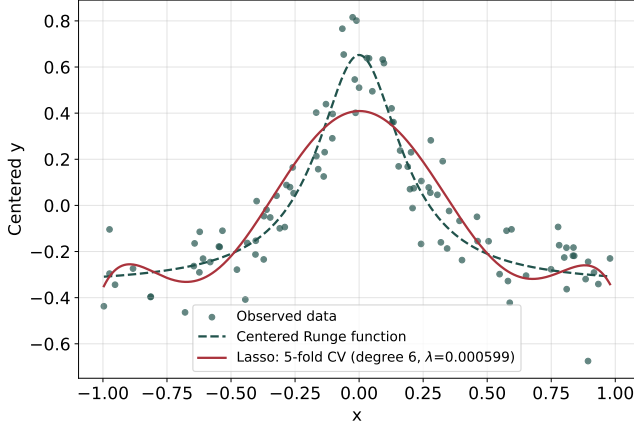


FIG. 40. Estimated solutions for the Runge function for polynomial degree 6, using the Lasso regression model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

OLS MSE decomposition, the variance does not rise in the same manner, but is kept at a value close to bias at larger polynomials. This closely resembles the observed behaviour for the 5-fold Ridge MSE decomposition displayed in Figure 32, with one caveat being that the variance stayed more tame. It seems that for higher degrees, the difference in variance decreases from degree to degree, and this may be said for any penalty parameter.

This gives us an estimated Runge function with optimal degree 6 and penalty parameter $\lambda = 0.000599$ for a noise standard deviation $\sigma = 0.1$, shown in Figure 40. For $\sigma = 0.01$, the estimated Runge function has an optimal degree of 10, with penalty parameter $\lambda = 1.0 \times 10^{-6}$ as shown in Figure 41. Increasing the data points in Figure 42, we again reach a higher polynomial degree, also with a lower penalty parameter and lower MSE as shown in Table VIII, similar to what we saw for the Ridge Regression with increased data points in Figure 37.

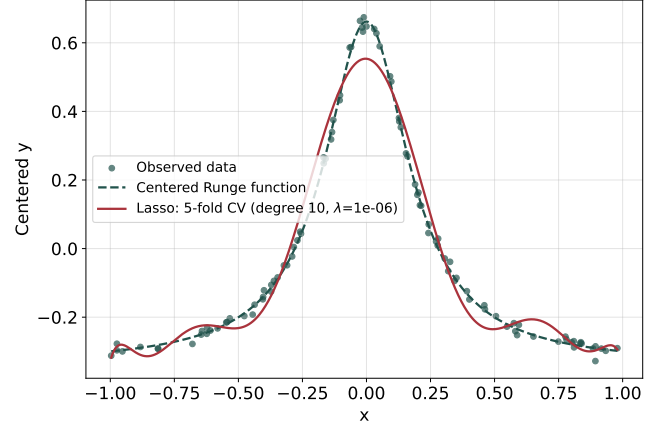


FIG. 41. Estimated solutions for the Runge function for polynomial degree 10, using the Lasso regression model with number of data points $n = 100$ and noise standard deviation $\sigma = 0.01$.

TABLE VI. Lasso regression, optimal values of the penalty parameter λ for different polynomial degrees using 5-fold cross-validation with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

Degree	λ
1	0.1
2	0.007743
3	0.02783
4	0.002154
5	0.007743
6	5.995×10^{-4}
7	5.995×10^{-4}
8	1.292×10^{-5}
9	4.642×10^{-5}
10	1.000×10^{-6}
11	1.000×10^{-6}
12	1.292×10^{-5}
13	1.292×10^{-5}
14	1.292×10^{-5}
15	1.292×10^{-5}
16	1.292×10^{-5}

TABLE VII. Lasso regression, optimal polynomial degree, corresponding penalty parameter λ , and MSE using 5-fold cross-validation with noise standard deviation $\sigma = 0.1$ and number of data points $n = 100$.

	Degree	λ	MSE
5-fold	6	5.995×10^{-4}	0.0280867

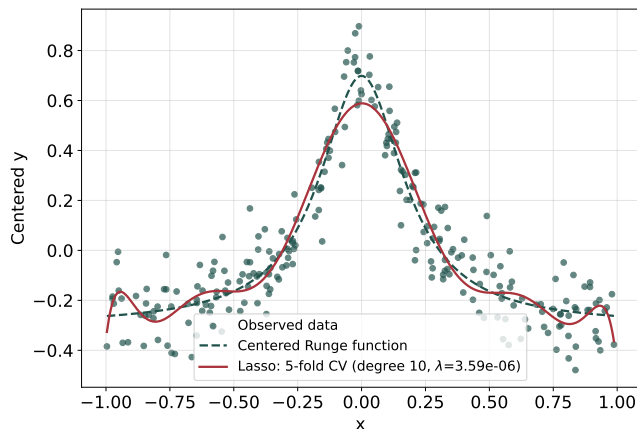


FIG. 42. Estimated solutions for the Runge function for polynomial degrees 10, using the Lasso regression model with noise standard deviation $\sigma = 0.1$ and number of data points $n = 250$.

TABLE VIII. Optimal polynomial degree, corresponding penalty parameter λ , and MSE using 5-fold cross-validation and noise standard deviation $\sigma = 0.1$ and number of data points $n = 250$.

	Degree	λ	MSE
5-fold	10	3.594×10^{-6}	0.0156956

V. CONCLUSIONS

In this paper, we have performed a comprehensive analysis of the Runge function, a classical test function, and the starting point for our analysis. We have implemented and tested different approaches for regression and gradient descent, as well as utilised various optimisers. Varying results, therefore, do not come as a surprise. In general, standard OLS, and Ridge without implementation of resampling methods provide lower values for the optimal polynomial coefficient (for instance 7 for the test data) compared to our obtained results when including gradient descent and optimiser algorithms. The bias-variance tradeoff analysis with bootstrap implementation is, however, the exception. For this case, an optimal polynomial degree of 6 was found. Upon implementing k-fold cross-validation, an optimal degree reaching the high value of 11 was obtained. Yet, as discussed in the paper, this might be explained by inconsistencies in data splitting into training and test data. These findings imply possible opportunities for improvement in the future.

For the penalty parameter λ , we also observe variation. Determining a general optimal value for λ presents a challenge; the optimal value will depend on the k-fold value, the learning rate γ , the different optimisers, and the method of regression. A general trend may be observed where penalty parameters often lie within $\lambda \in [10^{-2}, 1]$ across regression methods. In our final results, in [sec-](#)

[tion IV I](#), upon combining the multitude of concepts visited throughout the paper, we find penalty parameters down to the order of 10^{-6} for higher polynomial degrees. The learning rate is also varied in our analysis, posing the same challenge in its optimal estimation due to it being correlated to many parameters, among these the just mentioned λ . In our paper, we acknowledge the variant character of our results; we nevertheless end our analysis with a more conclusive set of results after having combined the different approaches and algorithms we have implemented. Among them is the k-fold cross-validation (for a range of k-folds values), the regression methods (OLS, Ridge, and Lasso), and the rigorous fitting attempts of the defined model to the Runge function. For this assortment of techniques, we obtain the following parameters: for OLS, we find the optimal polynomial degrees $\in [8, 8, 11]$ for k-fold values $\in [5, 10, 25]$. For Ridge, we obtain optimal polynomial degrees $\in [6, 6, 6]$ for k-fold values $\in [5, 10, 25]$, with corresponding values for the penalty parameter $\lambda \in [0.0433, 0.0486, 0.1385]$. For Lasso, we obtain an optimal polynomial degree of 6 for the k-fold value 5, with $\lambda = 0.000599$. We further convince ourselves that the fitting of our model to the Runge function does, in fact, provoke the famous Runge's phenomenon, inducing the characteristic oscillatory behaviour found at the edges of the interval where the function is defined.

Exploring additional combinations of parameters will be an important topic for further work. Investigating the effects of greater changes in the number of data points and the standard deviation for the noise would moreover be an interesting aspect to further diagnose in our understanding of the parameters' effect on our obtained results. We would henceforth be intrigued by drawing up the same analysis for new functions. We are highly optimistic about our analysis being a starting point for further sophisticated investigations of the Runge function.

-
- [1] Morten Hjort-Jensen. Lecturebook. <https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf>. MachineLearningUiO, accessed 22 September 2026.
 - [2] C. Runge. Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten. *Zeit. Math. Phys.*, 46:224–243, 1901.
 - [3] Interpolationcal. Runge’s Phenomenon: Why High-Degree Polynomials Fail. <https://interpolationcalc.com/blog/runges-phenomenon/>, June 2026. Accessed 23 September 2026.
 - [4] Morten Hjort-Jensen. Machine learning course materials. GitHub. <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc>. Accessed 4 October 2026.
 - [5] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
 - [6] Morten Hjort-Jensen. Week 38 exercises. GitHub. <https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/WeeklyMaterial/week38/exercisesweek38.ipynb>. Accessed 4 October 2026.
 - [7] Trevor Hastie, Jerome Friedman, and Robert Tibshirani. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer New York, New York, NY, 2001.
 - [8] James Bradbury et al. JAX: composable transformations of Python+NumPy programs, 2018.

Appendix A: Use of AI/LLM Tools

1. Tools used

- ChatGPT Version GPT-5.6 Luna Reasoning model from Open AI
- GPT UiO Version GPT-5.6 Terra
- Claude Code Version 2.1.285 Opus 5.5.
- Grammarly free version

2. Text writing and editing

In [section III](#) the pseudo codes written for our algorithms were inspired from pseudo code we asked Claude Code to generate. We however, ended up making a lot of changes it for it to fit what we wanted it to include. Assume this to be level 1.

In [section III C](#), we used Claude Code to help put forth how we should present the algorithm to precisely explain its function. Assume this to be level 1.

In [section III D](#) where Claude Code was used to help understand the consequence of the change to a L1 penalty parameter compared to the L2 one in OLS and Ridge. The [section III D](#) was still written ourselves, but asking Claude Code helped us with the understanding. Assume this to be level 1.

In overleaf some of the students have used Grammarly which checks correct spelling and helps with small sentence changes.

Beyond this, the report has level 0 of AI/LLM tool use for text writing and editing.

3. Code generation and assistance

In exercise g) we used the two tools described in [section A 1](#) to try to debug issues we had with the Lasso regression code. It helped us debug problems with no convergence. We would estimate our use to be a combination of level 1 and 2 in the category *Code Generation and Assistance* for this specific task. For this task the described tools furthermore helped us understand concepts associated with the exercise. The tools were however never used to generate text for our report.

[Table I](#) in [section IV E](#) is generated in ChatGPT from Open AI. For this purpose, the use is a level 2 in the category *Code Generation and Assistance*.

For general plotting code, we have used Chat GPT and GPT UiO (given in [section A 1](#)) to format at fix bugs in out plotting code. We would estimate our use to be a combination of level 1 and 2 in the category *Code Generation and Assistance*.